

实验报告

课程：数字逻辑与计算机组成实验

实验十二：计算机系统

姓名： 郑袭明

学号： 251502027

日期：2026 年 7 月 1 日

目录

1 系统概述	2
2 硬件：把外设接到 CPU 数据总线上	2
2.1 内存映射 I/O 的总体接入	2
2.2 接入 VGA 显示	3
2.3 接入 PS/2 键盘	4
2.4 接入定时器	4
3 软件：裸机系统与 Shell	4
3.1 显示与键盘底层驱动 (sys.c)	5
3.2 行编辑与输出系统 (io.c)	5
3.3 字符串工具 (str.c)	6
3.4 内存文件系统 (fs.c)	6
3.5 Shell (main.c)	7
4 实验流程	8
5 实验总结与思考	8
6 致谢	9

1 系统概述

这次实验要做的是一台能真正用起来的小计算机。它的核心是前面几次实验里搭好的那个 RV32I 五级流水线 CPU，本身已经支持完整的整数指令、数据前递、load-use 停顿，以及按字节、半字、字访存。但只有 CPU 还不够——它得能把东西显示出来，能接受输入，还得有个时间概念。于是这次给它接上了三样外设：VGA 字符显示、PS/2 键盘和一个秒级定时器，再在上面写一套裸机软件，让人可以坐在屏幕前敲命令。

文件	作用
<code>top.v</code>	顶层，连接 CPU、存储器与各外设
<code>rv32ip.v</code>	RV32I 五级流水线 CPU
<code>alu.v</code> 、 <code>regfile.v</code>	ALU 与寄存器堆
<code>dmem.v</code>	数据存储器，含字节/半字访存
<code>vga_char.v</code> 、 <code>vga_ctrl.v</code>	VGA 字符显存与扫描时序
<code>keyboard.v</code> 等	PS/2 键盘解析与扫描码转换
<code>timer.v</code>	秒级定时器

很多模块之前已经做完了，这里真正需要做的，是把这些外设挂到 CPU 的数据总线上，也就是 `top.v` 里那套内存映射 I/O 的接入逻辑。软件部分则是从头写的：一个带管道和重定向的 shell、一个内存文件系统、行编辑与输出系统、显示和键盘的底层驱动。

2 硬件：把外设接到 CPU 数据总线上

2.1 内存映射 I/O 的总体接入

CPU 对外只有一组数据存储器端口：地址 `dmemaddr`、写数据 `dmemdatain`、读数据 `dmemdataout`、写使能 `dmemwe`、访存类型 `dmemop`。读在时钟上升沿、写在下降沿。问题在于，现在挂在这组端口后面的不止一块 RAM，还有三个外设。所以现在将一组总线信号按地址分段。用地址的高 12 位 `dmemaddr[31:20]` 当作设备选择，不同的段对应不同的设备：

段（[31:20]）	基址	设备	读/写
0x001	0x00100000	数据 RAM	读写
0x002	0x00200000	VGA 字符缓冲	写
0x003	0x00300000	键盘寄存器	读
0x004	0x00400000	定时器	读写

写一侧的处理是把 `dmemwe` 按段拆开，每个设备各自得到一个只在本段地址下才有效的写使能，这样一次写绝不会同时落到两个设备上：

```
wire dmem_we = dmemwe & (dmemaddr[31:20] == 12'h001); // 数据 RAM
wire vga_we  = dmemwe & (dmemaddr[31:20] == 12'h002); // VGA
wire time_we = dmemwe & (dmemaddr[31:20] == 12'h004); // 定时器
```

读一侧则反过来，用一个多路选择器按地址段把对应设备的输出汇成 `dmemdataout` 送回 CPU：

```
assign dmemdataout = (dmemaddr[31:20] == 12'h001) ? dmem_out :
                    (dmemaddr[31:20] == 12'h003) ? key_reg  :
                    (dmemaddr[31:20] == 12'h004) ? time_reg :
                                                         32'b0 ;
```

剩下的就是在 `top.v` 里例化各外设，把地址、数据、时钟这些信号接到它们的端口上。

2.2 接入 VGA 显示

VGA 这块直接例化了两个模块：`vga_char` 管显存和字符渲染，`vga_ctrl` 管扫描时序和 RGB 输出。接入时，把上一节算出的 `vga_we`、地址 `dmemaddr[16:0]` 和写数据的低 8 位接到 `vga_char` 的写口；`vga_char` 输出的像素数据再接给 `vga_ctrl`，由它驱动板上的 VGA 引脚。像素时钟是 25 MHz，用一个时钟向导 IP `clk_wiz_0` 从 100 MHz 主时钟分频得到。

```
wire vga_we = dmemwe & (dmemaddr[31:20] == 12'h002);
vga_char vga (
    .wclk (dmemwrclk), .we (vga_we),
    .waddr(dmemaddr[16:0]), .wdata(dmemdatain[7:0]),
    .h_addr(h_addr), .v_addr(v_addr), .vga_data(vga_data)
);
```

每个字符单元的线性地址是“行左移 7 位再加列”，也就是行间距 128（实际只用前 80 列）。除了写字符，显存还藏着两个控制寄存器，靠写地址的高位来区分写的是哪一种：

```
if (waddr[16]) start_row <= wdata[4:0]; // 滚动寄存器：屏幕起始行
else if (waddr[15]) cursor_pos <= waddr[11:0]; // 光标位置寄存器
else vram[waddr[11:0]] <= wdata; // 普通写字符
```

也就是说，写 0x00210000 一带改的是滚动起点（用来整屏上滚），写 0x00208000 一带改的是光标位置，其余地址才是往显存里填字符。

2.3 接入 PS/2 键盘

键盘的 `keyboard` 模块有两个需要的输出：当前按键的 ASCII 码 `ascii_key`，以及一个每按一次键就加一的计数 `key_count`。接入要做的是把这两个值拼成一个 32 位字，在读时钟沿锁存进 `key_reg`，CPU 读 0x003 段时拿到的就是它：

```
keyboard kbd (  
    .clk(CLK100MHZ), .clrn(CPU_RESETN),  
    .ps2_clk(PS2_CLK), .ps2_data(PS2_DATA),  
    .key_count(key_count), .ascii_key(ascii_key)  
);  
reg [31:0] key_reg;  
always @(posedge dmemrdclk) key_reg <= {16'b0, key_count,  
    ascii_key};
```

这里 `key_count` 是关键。键盘没有缓冲队列，软件没法直接问“有没有新键”，于是用这个计数来判断：软件记住上一次读到的计数，发现计数变了就说明又按了一次键，这时 `ascii_key` 里的就是新字符。后面 `getch` 正是这么做的。另外，方向键、Home、End 这类键被映射成了约定的控制码（比如左方向对应 0x02），软件 `io.c` 里的按键定义就是按这套来的。

2.4 接入定时器

定时器最简单。例化已有的 `timer` 模块，把按段拆出来的写使能 `time_we` 和写数据接上去，它的输出 `time_reg` 已经并进了上面那个读多路里。`time_reg` 是一个以秒为单位的计数：模块内部在 100 MHz 下分频成 1 Hz 自增。软件往 0x004 段写一个值就能直接给它置数，也就是“对时”，这正是软件里 `time set HH:MM:SS` 的硬件依据。

3 软件：裸机系统与 Shell

软件这部分是这次从头写的，分几层：最底下是显示和键盘的驱动 (`sys.c`)，往上是行编辑和输出系统 (`io.c`)，再往上有字符串工具 (`str.c`) 和一个内存文件系统 (`fs.c`)，最上层是把这些拼起来的 shell (`main.c`)。整个程序是裸机运行的，没有操作系统也没有 C 标准库，所以进 `main` 之前还得自己用一小段汇编把栈指针摆好 (`main.c:9-13`)。下面按从底到上的顺序讲。

3.1 显示与键盘底层驱动 (sys.c)

驱动要做的就是将上一章那些硬件约定翻译成 C 里能调的函数。先把显存的三个地址用指针表示出来，对应显存、光标寄存器、滚动寄存器 (sys.c:3-5)：

```
char *vga_start = (char *)VGA_START;           // 0x00200000 字符缓冲
volatile char *vga_cursor = (volatile char *)VGA_CURSOR; // 光标
volatile int *vga_lineo = (volatile int *)VGA_LINE_O;  // 滚动
```

往屏幕打一个字符由 `putch` 完成 (sys.c:45-67)。它要分几种情况：回车换行就调 `new_line`；退格就把光标往回退一格并擦掉那个字符；普通字符就写进显存当前位置，再把列号加一，写满一行也自动换行。每动一次都要更新一下硬件光标的位置。

滚屏是这里比较有意思的地方。屏幕只有 30 行，写满之后不能停，得让内容往上滚。我的做法不是真的把显存整体搬一遍，而是改硬件的滚动寄存器：维护一个起始行 `start_line`，写满时让它往前挪一行并清掉新腾出来的那行，然后把新的起始行写进 `*vga_lineo` (sys.c:16-28)。硬件按这个起始行去取显存，看起来就是整屏向上滚了一行，而搬运的活完全交给了硬件。

读键盘是 `getch` (sys.c:69-80)。它就是不停地读键盘寄存器：

```
char getch(void) {
    static int last_count = 0;
    volatile unsigned int *key_reg = (volatile unsigned int *)
        KEY_START;
    while (1) {
        unsigned int w = *key_reg;
        int count = (int)((w >> 8) & 0xff);
        if (count != last_count) { // 计数变了，说明有新键
            last_count = count;
            return (char)(w & 0xff);
        }
    }
}
```

计数没变就一直转，一旦发现计数和上次不一样，就取出低 8 位的 ASCII 返回。

3.2 行编辑与输出系统 (io.c)

`getch` 只能拿到一个字符，但用户敲命令时希望能改、能移动光标。`read_line` (io.c:20-78) 就是为此写的一个完整行编辑器：支持左右移动、Home 跳到行首、End 跳到行尾、退格、Delete，以及在行中间插入字符。它维护两个量——缓冲里的内容长度

len，和光标当前所在的位置 pos。插入或删除字符时，要把后面的内容在缓冲里整体挪一格，再用 redraw 把光标之后的部分重画一遍，保证屏幕和缓冲始终一致。

这里还做了一件对 shell 很关键的事：输出重定向。平时 print 直接往屏幕写，但有时需要把一条命令的输出先攒到内存里（比如要喂给管道的下一段，或者要写进文件）。于是设了一个“输出去向”的开关（io.c:81-108）：

```
void out_redirect(char *buf, int cap) { /* 后续输出改写到 buf */ }
void out_restore(void) { /* 恢复成直接写屏幕 */ }
static void out_char(char c) {
    if (out_buf) { /* 写进缓冲并补 '\0' */ }
    else putchar(c); // 否则照常上屏
}
```

所有打印最终都走 out_char，所以只要切一下这个开关，同一套打印代码既能上屏也能写进缓冲。

打印接口本身也做得顺手一些。io.h 里用 C11 的 _Generic 按参数类型自动挑函数：字符串走 print_str，整数走 print_int，无符号走 print_uint，字符走 print_char；再配一组按参数个数展开的宏，支持一次传 1 到 10 个参数。于是上层可以直接写 println("x=", n) 这种混合类型的打印，算是一个不实现 printf 能做到的简化版本。

3.3 字符串工具 (str.c)

str.c 是一组小工具，没有标准库只能自己写：str_eq 比较两个字符串是否相等，str_to_i 把数字串转成整数，str_len 求长度，str_copy 做限长拷贝（防越界），str_contains 在一个串里找子串。它们被上层到处用到，比如命令分发要比较命令名，grep 要找子串。

3.4 内存文件系统 (fs.c)

文件系统是纯内存的，所有东西都放在 RAM 里，掉电就没了。它用一个固定 64 个节点的数组当存储，每个节点要么是目录要么是文件，靠三个下标 parent、first_child、next_sibling 串成一棵目录树，文件内容就存在节点里那块 256 字节的数据里（fs.c:9-22）：

```
typedef struct {
    char name[16];
    unsigned char used, is_dir;
    int parent, first_child, next_sibling; // 用下标串成树
    const char *content;
```

```
char data[256]; // 文件内容内联存放
} fs_node;
```

用“孩子—兄弟”表示法，每个目录只记第一个孩子，孩子之间用兄弟指针连成一条链，这样一棵任意宽度的树只用三个下标就能表达。

路径解析是 `fs_resolve` (`fs.c:73-98`)。它从根目录（路径以 `/` 开头）或当前目录出发，按 `/` 把路径切成一段段，逐段在当前目录的孩子里找名字匹配的节点；遇到 `.` 原地不动，遇到 `..` 回到父目录。创建文件或目录时还要先用 `split_parent` (`fs.c:162-182`) 把路径拆成“父目录”加“最后那一段名字”。

在这之上就是各个命令：`pwd` 顺着父链往上走再倒着打出来；`cd` 换当前目录；`ls` 列出一个目录下的孩子（目录名后加 `/`）；`cat` 打文件内容；`mkdir` 建目录；`rm` 删文件；`rmdir` 删空目录，并且会挡住删根目录、删当前目录、删非空目录这几种情况。给 shell 用的两个接口是 `fs_read` 和 `fs_write` (`fs.c:238-271`)：读返回内容指针；写支持新建、覆盖，也支持追加（`append`），分别对应重定向的 `>` 和 `>>`。

3.5 Shell (`main.c`)

最上层把前面这些拼成一个能交互的 shell。主循环很短：打印提示符 `$`，读一行，执行，循环 (`main.c:283-293`)。

所有命令集中登记在一张表里，用的是 X 宏的写法 (`main.c:123-163`)。命令名和处理函数只在 `COMMAND_LIST` 里写一遍，再分别展开成“注册”和“分发”两处代码，以后加命令只改这一个地方：

```
#define COMMAND_LIST(X) \
    X("hello", cmd_hello) X("clear", cmd_clear) \
    X("time", cmd_time)    X("echo", cmd_echo)    \
    X("grep", cmd_grep)    X("ls", cmd_ls) // ...
// 分发时再展开一次
#define X(name, fn) if (str_eq(cmd, name)) { fn(args); return; }
    COMMAND_LIST(X)
#undef X
```

一行命令真正的处理在 `run_command` (`main.c:230-281`)，它做了三件事。先按竖线 `|` 把整行切成若干段（最多 8 段），这就是管道；每一段先去掉首尾空格，再用 `parse_redir` (`main.c:176-225`) 把 `<`、`>`、`>>` 这些重定向解析出来；然后顺序执行每一段。段与段之间的数据传递用两块缓冲 `bufA` 和 `bufB` 轮流当中转：前一段的输出存进缓冲，作为后一段的输入 `g_stdin`，最后一段则直接输出到屏幕。如果某段带了 `>` 或 `>>`，就先把输出攒进 `filebuf`，执行完再写进文件系统。这套流程靠的正是 `io.c` 那个输出重定向开关。

内建命令本身都不复杂：`hello` 和 `clear` 一目了然；`time` 读出当前时间，`time set HH:MM:SS` 则用 `parse_hms` 解析后写定时器对时；`fibn` 迭代算第 `n` 个斐波那契数；`echo` 原样回显；`grep` 在输入 `g_stdin` 上逐行匹配，配合管道用。串起来就能敲出这样的命令行：

```
$ echo hello | grep he
hello
$ ls > a.txt
$ cat a.txt
```

第一条把 `echo` 的输出通过管道喂给 `grep`；第二条把 `ls` 的结果重定向写进文件；第三条再读出来。管道、重定向、文件系统在这里就连成了一体。

4 实验流程

软件这边在 `sys/software` 下 `make` 即可，生成 `main.coe` 和 `main_d.coe` 两份存储器初始化文件，分别给指令存储器和数据存储器用。硬件这边在 `sys/hardware` 下 `make`，生成 `top.bit`。把它们烧进板子后就能直接用。

这里稍微修改了提供的 `Makefile`，没有使用课程提供的虚拟机，也没有自己编译一遍 RISC-V 工具链，而是直接用本机的 `riscv64-unknown-elf-gcc`，再加上指定架构的 `-march=rv32i` 和 `-mabi=ilp32`，感觉比用非常重的虚拟机，或者自己处理编译的依赖要方便很多。

5 实验总结与思考

由于前面的几次实验已经完成了很大一部分内容，我也没有更换新的实现，所以这次实验的重点就只剩下怎么把不同的硬件连在一起，再加上软件部分了。不过软件部分与正常的开发有很大区别，比如没有标准库，所有的功能都需要自己写，还是有点麻烦的。

我在做硬件部分的时候没遇到什么问题，可能是最开始的框架搭的比较好，也没有多少需要改动的地方。软件部分倒是遇到了一些问题，比如在最开始写的时候，我的代码经常没法编译成功，这也让我想到了另一位同学遇到的问题：他没有把 `entry` 函数放在 `main.c` 的最顶部，导致了一直出现的问题；改掉以后立刻就看起来正常了。

实际上我觉得挺费时的是调这个光标，可能是因为我上个实验没有做这个。我最开始做的时候一直忘了在各种地方进行 `redraw`，导致显示出来的东西很诡异。要说我这个光标做的哪里不太好，我只能说，首先我为了性能好，把原本的字模大小从 `16x9` 改成了 `16x8`，而光标我做的比较简陋，显示的时候左右的间隙是不对称的，不太好看。其

次，这里我用了 Home 和 End 键，这个本来是需要专门处理长一点的控制码，但我偷懒了，直接把它们映射成了单个控制码，在现在看来没什么问题，但是实际上是一个不太好的实现，如果有机会我倒是希望做一个完整的键盘出来。

另外的几个遗憾我也正好一并说出来吧。首先我没有做一个完整的文件系统，只是在内存上实现了，当时想做的时候感觉接口真的太难写了，最后也没什么时间，就放弃了。第二个应该是需要在我的第一个基础上实现，就是我希望做可运行的二进制文件。我感觉这个其实是很好做的，就运行的时候把 PC 移过去，跑完了再回去就行，但是由于没有一个完整的文件系统，我做出来效果也不会好看，所以最后也没做。（其实我感觉这个我应该做一下的）最后一个就是支持一下浮点数了，我尝试了这个，但是真的好难写，硬件也要写很多，软件也要写很多，真的挺难的，最后我也没做出来。最开始想做的 GUI，由于我感觉要大改架构，也当然是没做出来了，只好无奈羡慕其他人。

所以最后也就完成了我一直想做的类似于 bash 的管道，和一个简单的文件系统来配合这个管道。感觉还是时间不太够，端午节感觉对我影响很大，加上我的课比较多，最后也没有做成我预期的效果，不过至少算是能看吧，完成了拓展内容，我也不能说做的很糟糕，至少还是有一点成就感的。

回顾整个实验的流程，我只能说，我似乎运气太好了一点，可能最开始所想的架构就不太容易出现问题的，在很多人碰到时序问题的时候我都没碰到，所以我在调试上花的时间大幅减少，主要集中在软件上，做了一些我想做的东西，也算是顺利吧。后来我也帮其他同学调试了一下，我只能说，这个实验真的好容易出问题，我能做成这样真的很幸运了。

我是和理论课一起选的这门，感觉两节课一起起到了相辅相成的作用，理论课上学的东西在这里能用上很多，我个人感觉也比只使用 Logisim 更好理解理论课的很多内容。这门课真的很有意思，虽然后面的几个实验都挺耗时间的，但做出来的成就感是远高于 Logisim 的。毕竟，谁不想做一个真实的计算机呢。

6 致谢

感谢李老师，解答了我的很多疑惑，在我遇到问题的时候也给了我很多帮助。也感谢南大，给我们带来了这门让我觉得有意思并也有收获的课程。

感谢和我一起做实验的几个同学，大家都很热情，做实验的时候我心情很愉快。感谢我的朋辈导师，他居然和我一起上课，这太巧了。感谢我的一位学长，他的博客指引我把理论课和实践课一起选了，不然我不会出现在这里。

感谢雪碧和巧乐兹，做实验的时候经常吃这些，超级好吃。感谢门尼和薛之谦，做实验的时候经常听他们，超级好听。

感谢 Vivado，感谢 Verilog，感谢 RISC-V，感谢所有的硬件工作者们。

感谢这个世界。